access the tail of the linked list in constant time, from which we recover the entry to delete: to locate it on the hash table and delete from it, we will need to hash the entry (or its ID) at an extra cost (potentially non-constant: for strings, it will depend on the length of the string).

Listing 7.3 LRU Cache evictOneEntry (private method)

```
function LRUCache::evictOneEntry()
  if hashTable.isEmpty() then
                                    #1
    return
false
         #2
  node ← elementsTail
  elementsTail =
node.previous()
#3
  if elementsTail != null
then                                                    #4
    elementsTail.next ← null

hashTable.delete(node.getKey())
              #5
  return
true
      #6
```

#1 Check that the cache is not empty.
#2 If it is, returns false to flag failure.
#3 Update the pointer to the least recent element. Invariant: if the cache is not empty, the tail is not null
#4 If the new tail is not null, updates its pointer to the next element (that must be `null` now, being the new tail).